

Enumeracion web con respuestas comodin (wildcard)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Algunas apps devuelven una pagina generica (200 OK) para toda ruta que no existe, en vez de un 404. Eso hace que gobuster/ffuf marquen como 'validas' todas las rutas y te llenen de falsos positivos. La clave es que las respuestas comodin comparten un tamano/estructura fijos, asi que filtras por ese tamano en lugar de por el codigo.

Un front controller o un handler catch-all sirve la misma plantilla (login, home, error bonito) para rutas desconocidas y responde 200. El fuzzer solo mira el status por defecto, asi que no ve la diferencia hasta que le dices que ignore ese tamano concreto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un escaneo de directorios devuelve casi todo en 200 con longitudes identicas, o cuando abrir una ruta inventada a mano te da la misma pagina que una ruta real. Sospecha en apps con enrutado propio (PHP con .htaccess, SPA, frameworks).

Cómo identificarla

- Casi todas las lineas del escaneo son 200 y comparten la misma longitud.
- Una ruta inventada a mano devuelve la misma pagina que el sitio real.
- Tras encontrar login.php, buscar backups con '-x bak' sobre la palabra 'login' prueba login.bak pero no login.php.bak: repite el fuzzing sobre los nombres COMPLETOS ya descubiertos (login.php.bak/.old/.save).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pide una ruta imposible (p.ej. /qwerty-noexiste-123) y anota su status y bytes; si es 200 con un tamano fijo, filtra ese tamano en el fuzzer.

Evidencia esperada: El escaneo pasa de cientos de falsos 200 a una lista corta de rutas realmente distintas.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
gobuster dir -u $URL/ -w /usr/share/wordlists/dirb/common.txt -x php,bak,old,txt --exclude-length 1491
```

Objetivo y contexto: `--exclude-length` descarta las respuestas comodín de ese tamaño fijo (aquí 1491 bytes), dejando solo lo que difiere de verdad. Sustituye 1491 por el tamaño que midas.

Resultado esperado: Una lista corta de rutas con longitud distinta a la comodín. Ahí están los ficheros reales (login.php, api_login.php, etc.).

Capa 2: prueba documentada

```
ffuf -u $URL/login.phpFUZZ -w <(printf '%s\n' .bak .old .save .swp ~) -mc all -fs 1491
```

Objetivo y contexto: Segunda pasada sobre un fichero YA descubierto para encontrar copias por extensión del nombre completo, el fallo que `'-x bak'` sobre la palabra base no cubre.

Resultado esperado: login.php.bak/.old/.save si existen. Un backup servido como texto = código fuente expuesto con credenciales y endpoints.

Anatomía de los payloads

La primera prueba es `gobuster dir -u $URL/ -w /usr/share/wordlists/dirb/common.txt -x php,bak,old,txt --exclude-length 1491`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El escaneo pasa de cientos de falsos 200 a una lista corta de rutas realmente distintas.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Mide la respuesta comodín pidiendo una ruta imposible y apunta bytes/palabras.
- Relanza el fuzzer excluyendo ese tamaño para eliminar los falsos positivos.
- Sobre los ficheros reales descubiertos, haz una segunda pasada buscando copias por extensión del nombre completo (.php.bak, .php.old, .php.save, .swp).

- Abre los backups: se sirven como texto y suelen filtrar credenciales, endpoints reales y logica (fuente expuesta).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor devuelve 404 real para rutas inexistentes (el filtrado por status normal ya sirve).
- Las respuestas comodín tienen longitud variable (contenido dinámico): filtra entonces por regex/palabras, no por bytes fijos.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El servidor responde 200 a CUALQUIER ruta, así que el código HTTP no distingue lo real de lo inexistente: filtra por longitud o palabras, no por status. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (enum-wildcard-responses)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.